



**Fatec
Votorantim**

**BANCO DE DADOS APLICADO
À CIÊNCIA DE DADOS I**

Profº Me. Jones Artur Gonçalves



Subconsultas

NOÇÕES BÁSICAS SOBRE SUBCONSULTAS

Uma subquery ou subconsulta é quando o resultado de uma consulta é utilizado por outra consulta, de forma encadeada e contida no mesmo comando SQL.

Restrições:

- A query interna deve ser usada dentro de parênteses
- A subquery não pode conter a cláusula order by
- Quando utilizando múltiplas colunas em uma subquery, estas colunas devem aparecer na mesma ordem e com os mesmos tipos de dados da query principal, além do mesmo número de colunas
- Podem ser utilizados operadores lógicos



NOÇÕES BÁSICAS SOBRE SUBCONSULTAS

- Uma subconsulta é uma consulta aninhada em uma instrução SELECT, INSERT, UPDATE ou DELETE.
- Uma subconsulta pode ser usada em qualquer lugar em que é permitida uma expressão
- Uma subconsulta também é chamada de uma consulta interna ou seleção interna, enquanto a instrução que contém uma subconsulta também é chamada de uma consulta externa ou seleção externa

NOÇÕES BÁSICAS SOBRE SUBCONSULTAS

- Muitas instruções Transact-SQL que incluem subconsultas podem ser alternativamente formuladas como junções
- Outras perguntas só podem ser feitas com subconsultas
- Em Transact-SQL, normalmente não há nenhuma diferença de desempenho entre uma instrução que inclui uma subconsulta e uma versão equivalente semanticamente que não inclui
- Entretanto, em alguns casos em que a existência deve ser verificada, uma junção tem um desempenho melhor

NOÇÕES BÁSICAS SOBRE SUBCONSULTAS

- Uma subconsulta aninhada na instrução `SELECT` externa tem os seguintes componentes:
 - Uma consulta regular `SELECT` incluindo os componentes regulares de lista de seleção
 - Uma cláusula regular `FROM` que inclui um ou mais nomes de tabela ou de exibição
 - Uma cláusula `WHERE` opcional
 - Uma cláusula `GROUP BY` opcional
 - Uma cláusula `HAVING` opcional



NOÇÕES BÁSICAS SOBRE SUBCONSULTAS

- A consulta SELECT de uma subconsulta sempre é incluída em parênteses
- Uma subconsulta pode ser aninhada na cláusula WHERE ou HAVING de uma instrução externa SELECT, INSERT, UPDATE ou DELETE ou em outra subconsulta
- Até 32 níveis de aninhamento são possíveis, embora o limite varie com base na memória disponível e na complexidade de outras expressões da consulta
- Uma subconsulta pode aparecer em qualquer lugar em que uma expressão possa ser usada, se retornar um único valor

NOÇÕES BÁSICAS SOBRE SUBCONSULTAS

Existem três tipos principais de subconsultas:

Subconsulta Correlacionada:

Uma subconsulta correlacionada é aquela que depende de valores da consulta principal ou de uma consulta externa. Ela é executada para cada linha da consulta principal. *Em outras palavras, os resultados da subconsulta são afetados pelos valores da linha atual da consulta principal.* Esse tipo de subconsulta é usado quando você precisa combinar informações de várias tabelas com base em critérios específicos.

Exemplo: Suponha que você deseja encontrar todos os alunos que têm notas superiores à média de suas respectivas turmas. A subconsulta depende da turma atual na consulta principal para calcular a média daquela turma e, em seguida, compara as notas dos alunos com essa média.

NOÇÕES BÁSICAS SOBRE SUBCONSULTAS

Subconsulta Escalar:

Uma subconsulta escalar é aquela que retorna um único valor ou resultado. Ela é usada para obter informações específicas que podem ser usadas em operações na consulta principal. Por exemplo, você pode usar uma subconsulta escalar para obter o número total de produtos em uma categoria específica.

Exemplo: Para calcular o preço médio de todos os produtos em uma categoria, você pode usar uma subconsulta escalar que retorna a média dos preços dos produtos naquela categoria.

NOÇÕES BÁSICAS SOBRE SUBCONSULTAS

Subconsulta de Tabela:

Uma subconsulta de tabela é aquela que retorna um conjunto de resultados, semelhante a uma tabela. Ela pode ser usada em cláusulas como IN, EXISTS, ANY, ALL ou JOIN para filtrar ou combinar dados na consulta principal. Essa subconsulta é usada para comparar conjuntos de dados em diferentes tabelas.

Exemplo: Suponha que você deseja encontrar todos os produtos que foram pedidos em uma lista específica de pedidos. Você pode usar uma subconsulta de tabela para obter os IDs de produtos associados a esses pedidos e, em seguida, compará-los com os produtos na consulta principal.

Esses são os três principais tipos de subconsultas no SQL, e cada um tem seu próprio propósito e uso. As subconsultas são poderosas ferramentas para recuperar, filtrar e combinar dados de diferentes maneiras em um banco de dados.

Consulta
Externa

```
SELECT *  
FROM LIVRO  
WHERE PRECO IN
```

Consulta
Interna

```
(SELECT preco  
FROM LIVRO  
WHERE TO_CHAR(datapublicacao,'YYYY') < '1998')
```

TITULO	DATAPUBLICACAO	CODLIVRO	NUMPAGS	PRECO	CODEDITORA	SIGLA
PROGRAMANDO EM LINGUAGEM C	01/10/1997	2	47	40,33	1	P
REDES DE COMPUTADORES	01/09/1996	5	93	46,42	2	R

OPERADORES LÓGICOS

Operador	Significado
ALL	TRUE se tudo em um conjunto de comparações for TRUE.
AND	TRUE se as duas expressões booleanas forem TRUE.
ANY	TRUE se qualquer conjunto de comparações for TRUE.
BETWEEN	TRUE se o operando estiver dentro de um intervalo.
EXISTS	TRUE se uma subconsulta tiver qualquer linha.
IN	TRUE se o operando for igual a um de uma lista de expressões.
LIKE	TRUE se o operando corresponder a um padrão.
NOT	Inverte o valor de qualquer outro operador booleano.
OR	TRUE se qualquer expressão booleana for TRUE.
SOME	TRUE se algum conjunto de comparações for TRUE.



USANDO ALL EM UMA SUBCONSULTA

O comando **ALL** em SQL é uma forma de fazer comparações em consultas, especialmente em subconsultas.

Ele é usado para comparar um valor com todos os valores retornados por uma subconsulta.

Em outras palavras, o **ALL permite verificar se uma condição é verdadeira para todos os resultados de uma subconsulta.**

Em resumo, o comando **ALL** é uma ferramenta útil para fazer comparações complexas em SQL, onde você deseja que uma condição seja verdadeira para todos os resultados de uma subconsulta. Ele ajuda a encontrar os registros que atendem a critérios específicos em um conjunto de dados.

USANDO ALL EM UMA SUBCONSULTA

Suponha que temos uma tabela de produtos e queremos encontrar todos os produtos que têm um preço maior do que todos os outros produtos. Usar o **ALL** nos permite fazer essa comparação.

```
SELECT NomeProduto FROM Produto
```

```
WHERE Preço > ALL (SELECT Preço FROM Produto Where NomeProduto =  
'Chapéu')
```

Neste exemplo, a subconsulta (**SELECT Preço FROM Produto Where NomeProduto = 'Chapéu'**) retorna o preço do produto chapéu na tabela.

Usando **Preço > ALL**, estamos verificando se o preço de cada produto na tabela é maior do que o preço retornado pela subconsulta. Se isso for verdadeiro para todos os produtos, o produto será selecionado na consulta principal.

Exemplos de consultas usando a cláusula **ALL** em SQL. A cláusula **ALL** é usada para comparar uma expressão com todos os valores retornados por uma subconsulta. Aqui estão os exemplos:

Neste exemplo, estamos selecionando todos os produtos cujo preço é maior do que o preço de todos os produtos da categoria 'Eletrônicos':

```
SELECT NomeProduto
```

```
FROM Produto
```

```
WHERE Preco > ALL (SELECT Preco FROM Produto
```

```
WHERE CategoriaID = (SELECT CategoriaID FROM Categoria
```

```
WHERE NomeCategoria = 'Eletrônicos'));
```

USANDO SOME / ANY EM UMA SUBCONSULTA

- Tanto ANY quanto SOME são utilizados para comparar valores em subconsultas com valores em uma consulta principal. **Eles permitem verificar se uma determinada condição é verdadeira para pelo menos um dos resultados retornados pela subconsulta.**
- SOME e ANY são equivalentes
- Precisam de um operador do tipo =, <>, >, <, <= ou >= antes de ANY e SOME na nossa consulta.
- Em resumo, tanto ANY quanto SOME são usados para comparar um valor com pelo menos um dos resultados de uma subconsulta. Eles são úteis quando você deseja verificar se uma condição é verdadeira para pelo menos um item em um conjunto de dados.

Comando ANY

- O comando ANY é usado para comparar um valor com qualquer um dos valores retornados pela subconsulta. Se a condição for verdadeira para pelo menos um dos resultados da subconsulta, a linha é incluída no resultado da consulta principal.
- *Por exemplo, suponha que você queira listar todos os produtos cujo preço seja menor do que o preço de pelo menos um produto da categoria "Eletrônicos", cujo o código é 1 no nosso exemplo. A consulta ficaria assim:*

```
SELECT NomeProduto
```

```
FROM Produto
```

```
WHERE Preço < ANY (SELECT Preço FROM Produto WHERE CategoricalID = 1);
```

- Isso retornaria todos os produtos cujo preço é menor do que pelo menos um produto da categoria "Eletrônicos."

Comando SOME

- O comando SOME funciona da mesma maneira que ANY. Ele é usado para comparar um valor com qualquer um dos valores retornados pela subconsulta. A diferença é que SOME é uma alternativa ao ANY e pode ser usado de forma intercambiável.
- Aqui está um exemplo similar usando SOME:

```
SELECT NomeProduto
```

```
FROM Produto
```

```
WHERE Preço < SOME (SELECT Preço FROM Produto WHERE CategoricalID = 1);
```

- Este comando também retornaria todos os produtos cujo preço é menor do que pelo menos um produto da categoria "Eletrônicos".

USANDO EXISTS EM SUBCONSULTAS

- O comando EXISTS é usado para verificar se existem registros em uma subconsulta que atendem a uma determinada condição. Ele retorna verdadeiro se pelo menos um registro for encontrado na subconsulta e falso se nenhum registro atender à condição.
- Em resumo, o comando EXISTS é usado para verificar se há registros em uma subconsulta que atendem a uma determinada condição. É útil quando você deseja determinar se pelo menos um registro corresponde a uma condição específica. Se pelo menos um registro existir, a condição do EXISTS será verdadeira.

USANDO EXISTS EM SUBCONSULTAS

- *Suponha que temos duas tabelas, uma para os produtos e outra para os pedidos, e queremos encontrar todos os produtos que foram pedidos pelo menos uma vez. Podemos usar o EXISTS para fazer isso:*

```
SELECT NomeProduto
FROM Produto
WHERE EXISTS (SELECT * FROM Pedido
              WHERE Pedido.ProdutoID = Produto.ProdutoID);
```

- Neste exemplo, a subconsulta (**SELECT * FROM Pedido WHERE Pedido.ProdutoID = Produto.ProdutoID**) verifica se há pelo menos um registro na tabela "Pedido" que corresponda ao ProdutoID na tabela "Produto". Se for encontrado pelo menos um registro, o EXISTS retornará verdadeiro e o produto será incluído na consulta principal.



USANDO SUBCONSULTAS COM IN

- O comando IN é usado para verificar se um valor específico está presente em um conjunto de valores retornados por uma subconsulta. Ele retorna verdadeiro se o valor estiver contido na lista de valores da subconsulta e falso caso contrário.
- Em resumo, o comando IN é usado para verificar se um valor específico está presente em um conjunto de valores retornados por uma subconsulta.
- É útil quando você deseja selecionar registros que correspondam a uma lista específica de valores ou critérios. Se o valor estiver contido na lista, o IN retornará verdadeiro e o registro será incluído na consulta principal.

USANDO SUBCONSULTAS COM IN

- *Suponha que temos uma tabela de produtos e queremos encontrar todos os produtos que estão nos pedidos 1 e 3. Podemos usar o IN para fazer isso:*

```
SELECT NomeProduto
```

```
FROM Produto
```

```
WHERE ProdutoID IN (SELECT ProdutoID FROM Pedido WHERE PedidoID IN (1, 3));
```

- Neste exemplo, a subconsulta interna (SELECT ProdutoID FROM Pedido WHERE PedidoID IN (1, 3)) retorna os IDs dos produtos que foram pedidos nos pedidos 1 e 3. A consulta principal seleciona os nomes dos produtos cujos IDs estão presentes nessa lista.

USANDO SUBCONSULTAS

1. Listar os nomes dos produtos que pertencem à categoria 'Eletrônicos':

```
SELECT NomeProduto  
FROM Produto  
WHERE CategoriaID = (SELECT CategoriaID FROM Categoria  
WHERE NomeCategoria = 'Eletrônicos');
```

2. Listar os nomes dos produtos comprados em pedidos realizados em janeiro de 2023:

```
SELECT NomeProduto  
FROM Produto  
WHERE ProdutoID IN (SELECT ProdutoID FROM Pedido  
WHERE YEAR(DataPedido) = 2023 AND MONTH(DataPedido) = 1);
```

USANDO SUBCONSULTAS

3. Listar os nomes das categorias que possuem produtos com preços acima de 100 reais:

```
SELECT NomeCategoria  
FROM Categoria  
WHERE CategoriaID IN (SELECT CategoriaID FROM Produto WHERE Preco > 100);
```

4. Listar os nomes de produtos que ainda não foram pedidos

```
SELECT NomeProduto  
FROM Produto  
WHERE ProdutoID NOT IN (SELECT ProdutoID FROM Pedido);
```


USANDO SUBCONSULTAS

5. Listar os nomes dos produtos que foram comprados em algum pedido:

```
SELECT NomeProduto
```

```
FROM Produto
```

```
WHERE ProdutoID IN (SELECT ProdutoID FROM Pedido);
```

6. Listar os nomes das categorias que têm pelo menos 2 produtos:

```
SELECT NomeCategoria
```

```
FROM Categoria
```

```
WHERE CategoriaID IN (SELECT CategoriaID FROM Produto GROUP BY  
CategoriaID HAVING COUNT(*) >= 2);
```

USANDO SUBCONSULTAS

7. Listar os nomes dos produtos que pertencem a categorias cujos nomes começam com 'E':

```
SELECT NomeProduto
FROM Produto
WHERE CategoriaID IN (SELECT CategoriaID FROM Categoria
                      WHERE NomeCategoria LIKE 'E%');
```

8. Listar os nomes das categorias que têm produtos comprados em pedidos realizados em 2023:

```
SELECT NomeCategoria
FROM Categoria
WHERE CategoriaID IN (SELECT CategoriaID FROM Produto WHERE ProdutoID IN (SELECT
ProdutoID FROM Pedido WHERE YEAR(DataPedido) = 2023));
```

USANDO SUBCONSULTAS

9. Listar os nomes dos produtos cujo preço é o mais alto entre todos os produtos:

```
SELECT NomeProduto  
FROM Produto  
WHERE Preço = (SELECT MAX(Preço) FROM Produto);
```

10. Listar os nomes dos produtos que não foram comprados em pedidos realizados em janeiro de 2023:

```
SELECT NomeProduto  
FROM Produto  
WHERE ProdutoID NOT IN (SELECT ProdutoID FROM Pedido WHERE  
YEAR(DataPedido) = 2023 AND MONTH(DataPedido) = 1);
```

USANDO SUBCONSULTAS

11. Listar todos os pedidos de produtos da categoria 'Eletrônicos'

```
SELECT p.PedidoID, p.DataPedido
```

```
FROM Pedido p
```

```
WHERE p.ProdutoID IN (SELECT ProdutoID FROM Produto WHERE CategoricalID = 1);
```

12. Listar os nomes dos produtos e suas categorias em pedidos

```
SELECT pr.NomeProduto, c.NomeCategoria
```

```
FROM Produto pr
```

```
JOIN Categoria c ON pr.CategoricalID = c.CategoricalID
```

```
WHERE pr.ProdutoID IN (SELECT ProdutoID FROM Pedido);
```

USANDO SUBCONSULTAS

13. Contar o número de produtos em cada categoria

```
SELECT c.NomeCategoria, (SELECT COUNT(*) FROM Produto  
WHERE CategoricalID = c.CategoricalID) AS NumeroProdutos  
FROM Categoria c;
```

14. Listar os produtos em pedidos feitos em fevereiro de 2023

```
SELECT pr.NomeProduto  
FROM Produto pr  
WHERE pr.ProdutoID IN (SELECT ProdutoID FROM Pedido WHERE  
MONTH(DataPedido) = 2 AND YEAR(DataPedido) = 2023);
```

OPERAÇÕES DE CONJUNTO

- **União**
- **Intersecção**
- **Subtração**

- **Exemplo:**
 - Sejam os conjuntos:
 - $A = \{1, 2, 3, 4, 5, 6\}$
 - $B = \{5, 6, 7, 8, 9\}$
 - $A \cup B = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$
 - $A \cap B = \{5, 6\}$
 - $A - B = \{1, 2, 3, 4\}$
 - $B - A = \{7, 8, 9\}$

UNION E UNION ALL

Combina os resultados de duas ou mais consultas em um único conjunto de resultados, que inclui todas as linhas pertencentes a todas as consultas da união

A operação UNION e UNION ALL são diferentes de usar junções que combinam colunas de duas tabelas

Regras básicas de combinação dos conjuntos de resultados de duas consultas usando UNION:

- O número e a ordem das colunas devem ser iguais em todas as consultas
- Os tipos de dados devem ser compatíveis.

Em resumo, **UNION** e **UNION ALL** são usados para combinar resultados de diferentes consultas e criar uma única lista. **UNION** remove registros duplicados, enquanto **UNION ALL** inclui todos os registros, inclusive duplicados, no resultado final. Isso pode ser útil quando você deseja combinar e apresentar informações de várias fontes em uma única lista.



UNION e **UNION ALL** são usados para combinar os resultados de duas ou mais consultas em uma única lista. Aqui está como eles funcionam:

UNION:

Remove registros duplicados: Quando você usa **UNION**, ele combina os resultados de diferentes consultas e remove registros duplicados. Isso significa que apenas registros únicos são retornados no resultado final.

UNION ALL:

Inclui todos os registros, inclusive duplicados: Ao contrário do **UNION**, o **UNION ALL** combina os resultados de diferentes consultas, mas não remove registros duplicados. Isso significa que todos os registros, inclusive os duplicados, são retornados no resultado final.

Exemplo 1: UNION

Suponha que queremos listar todas as categorias que têm produtos com preços acima de 1000 ou produtos que foram pedidos. Podemos usar UNION para fazer isso:

```
SELECT NomeCategoria
```

```
FROM Categoria
```

```
WHERE CategoriaID IN (SELECT CategoriaID FROM Produto WHERE Preço > 1000)
```

```
UNION
```

```
SELECT NomeCategoria
```

```
FROM Categoria
```

```
WHERE CategoriaID IN (SELECT Produto.CategoriaID FROM Produto JOIN Pedido  
ON Produto.ProdutoID = Pedido.ProdutoID);
```

Exemplo 2: UNION ALL

Suponha que agora queremos listar todos os nomes de produtos que custam menos de 100 ou produtos que custam mais de 1000. Podemos usar UNION ALL para fazer isso e incluir todos os produtos, mesmo que haja produtos duplicados:

```
SELECT NomeProduto
```

```
FROM Produto
```

```
WHERE Preço < 100
```

```
UNION ALL
```

```
SELECT NomeProduto
```

```
FROM Produto
```

```
WHERE Preço > 1000;
```

Neste caso, estamos listando os nomes de produtos que custam menos de 100 e os nomes de produtos que custam mais de 1000, incluindo produtos duplicados, se houver.

INTERSECT E EXCEPT

INTERSECT e **EXCEPT** são usados para comparar os resultados de duas consultas e obter conjuntos de registros com base em critérios específicos. Aqui está como eles funcionam:

os de registros com base em critérios específicos. Aqui está como eles funcionam:

INTERSECT:

Retorna registros que existem em ambas as consultas: O **INTERSECT** é usado para encontrar registros que estão presentes nas duas consultas, ou seja, **ele retorna a interseção dos resultados das consultas.**

EXCEPT:

Retorna registros que existem apenas em uma das consultas: O **EXCEPT** é usado para encontrar registros que existem em uma consulta, mas não na outra. **Ele retorna os registros que estão presentes na primeira consulta e ausentes na segunda.**



INTERSECT E EXCEPT

Em resumo, INTERSECT é usado para encontrar registros que existem em ambas as consultas, enquanto EXCEPT é usado para encontrar registros que existem em uma consulta, mas não na outra.

Eles são úteis quando você deseja comparar conjuntos de registros entre consultas para obter informações específicas sobre as diferenças ou semelhanças.

Exemplo 1: INTERSECT

Suponha que queremos encontrar todas as categorias que têm produtos com preços superiores a 1000 e que também têm produtos pedidos. Podemos usar INTERSECT para fazer isso:

```
SELECT NomeCategoria
```

```
FROM Categoria
```

```
WHERE CategoriaID IN (SELECT CategoriaID FROM Produto WHERE Preço > 1000)
```

```
INTERSECT
```

```
SELECT NomeCategoria
```

```
FROM Categoria
```

```
WHERE CategoriaID IN (SELECT Produto.CategoriaID FROM Produto JOIN Pedido ON  
Produto.ProdutoID = Pedido.ProdutoID);
```

Exemplo 2: EXCEPT

Suponha que agora queremos encontrar todas as categorias que têm produtos com preços superiores a 100, mas que não têm produtos pedidos. Podemos usar EXCEPT para fazer isso:

```
SELECT NomeCategoria
```

```
FROM Categoria
```

```
WHERE CategoriaID IN (SELECT CategoriaID FROM Produto WHERE Preço > 100)
```

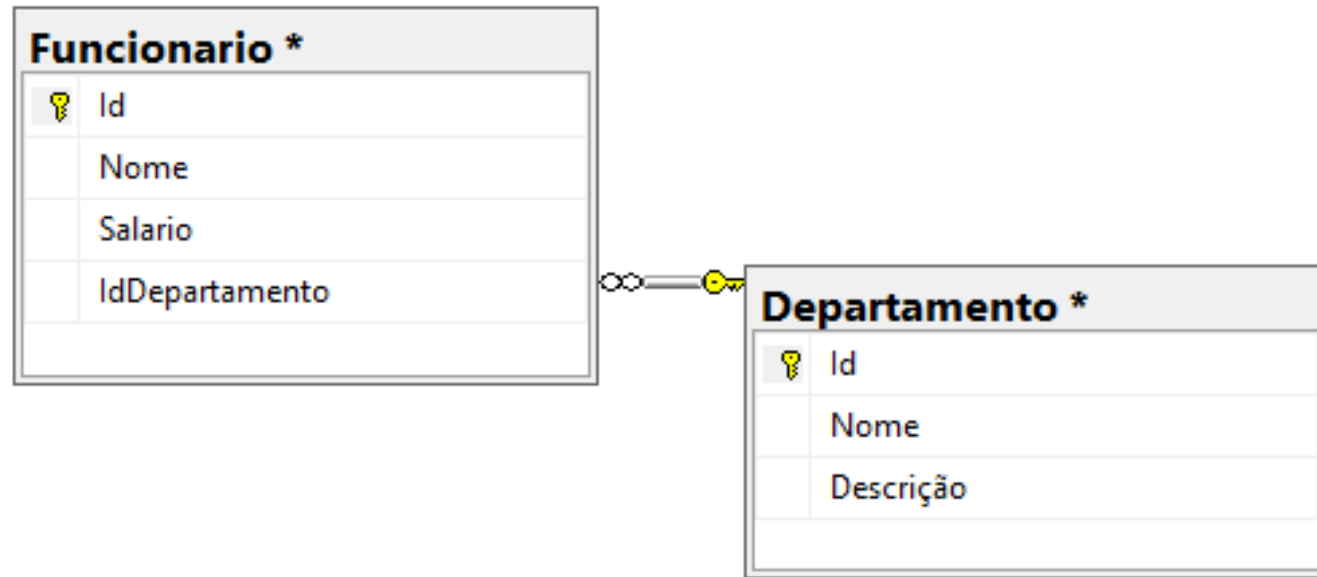
```
EXCEPT
```

```
SELECT NomeCategoria
```

```
FROM Categoria
```

```
WHERE CategoriaID IN (SELECT Produto.CategoriaID FROM Produto JOIN Pedido ON  
Produto.ProdutoID = Pedido.ProdutoID);
```

Junção



Junção x Subconsulta

```
-- retorna o nome dos funcionários do departamento de TI
```

```
// Subquery
```

```
SELECT NOME FROM FUNCIONARIO WHERE IDDEPARTAMENTO IN  
(SELECT ID FROM DEPARTAMENTO WHERE NOME = 'TI')
```

```
// Inner Join
```

```
SELECT FUNCIONARIO.NOME FROM DEPARTAMENTO  
INNER JOIN FUNCIONARIO  
ON DEPARTAMENTO.ID = FUNCIONARIO.IDDEPARTAMENTO  
WHERE DEPARTAMENTO.NOME = 'TI'
```


BIBLIOGRAFIA



BÁSICA:

DATE, C. J. PROJETO DE BANCO DE DADOS E TEORIA RELACIONAL: FORMAS NORMAIS E TUDO O MAIS. SÃO PAULO: NOVATEC, 2015.

KORTH, HENRY F.; SUDARSHAN, S.; SILBERSCHATZ, ABRAHAM. SISTEMA DE BANCO DE DADOS. 7 ED. SÃO PAULO: GEN; LTC, 2020.

SILVA, RICK. CURSO INTENSIVO DE MYSQL: UMA INTRODUÇÃO PRÁTICA AO DESENVOLVIMENTO BASEADO EM BANCO DE DADOS. RIO DE JANEIRO: NOVATEC, 2023.

COMPLEMENTAR:

NIELD, Thomas. Introdução à Linguagem SQL: abordagem prática para iniciantes. Rio de Janeiro: Novatec, 2016. NADEAU, Tom; LIGHTSTONE, Sam S.; TEOREY, Toby J.; JAGADISH, H. V. Projeto e Modelagem de Banco de Dados. 2 ed. Rio de Janeiro: Elsevier; Campus, 2013.



OBRIGADO